ICADEP-XXI
INSTITUTO DE CAPACITACIÓN Y DESARROLLO PROFESIONAL SIGLO XXI

MÓDULO 7 · DOCUMENTO DE ESTUDIO

Automatización con macros y VBA.

Cómo convertir tareas repetitivas de Excel en procesos rápidos, confiables y de un solo clic.

Por: **Jarlin Figueroa, MA** · Instituto ICADEP-XXI

CONTENIDO

1 El caso del módulo
2 ¿Qué es automatizar?
3 Macros y VBA
4 Preparar Excel
5 Grabar macros
6 El editor de VBA
7 Estructura de una macro
8 El modelo de objetos
9 Trabajar con celdas
10 Variables y tipos de datos
11 Decisiones: If...Then...Else
12 Repeticiones: For y For Each
13 Hojas y fórmulas desde VBA
14 MsgBox e InputBox
15 Botones y depuración
16 Caso integrador
17 Buenas prácticas y aplicación
18 Glosario
· Referencias

Objetivo del módulo: desarrollar automatizaciones básicas mediante macros y VBA para mejorar la eficiencia y reducir tareas manuales en Excel. Requiere Excel de escritorio para Windows (2021 o Microsoft 365): las macros no funcionan en Excel para la web.

CAPÍTULO 1

El caso del módulo: el informe mensual.

En una institución del sector salud, un analista prepara cada mes un informe de reclamaciones. El proceso siempre es el mismo y consume horas de trabajo manual.

Cada mes, el analista recibe la información de las reclamaciones presentadas por las Administradoras de Riesgos de Salud (ARS) y debe convertirla en un reporte para la dirección. Para lograrlo repite, una y otra vez, las mismas tareas:

1. **Organizar la información** que llega de las ARS en una hoja de trabajo.
2. **Aplicar formatos**: títulos, colores, formato de moneda y bordes.
3. **Actualizar cálculos**: cuántas reclamaciones y cuánto monto tiene cada ARS.
4. **Preparar la hoja de resumen** que se presenta a la dirección.

Cuando estas tareas se hacen a mano, mes tras mes, aparecen dos problemas: el **tiempo** que se consume y los **errores** que se cuelan (una fila que no se suma, un formato distinto al del mes anterior, un dato copiado en la celda equivocada).

> **EL RETO**
> La institución desea automatizar parte de este proceso para reducir el tiempo de trabajo y minimizar errores. Las **macros** y el lenguaje **VBA** permiten lograrlo: el procedimiento se define una vez y luego Excel lo ejecuta con un solo clic.



Figura 1. Las cuatro tareas del informe se reúnen en una sola macro que se ejecuta con un botón.

A lo largo de este documento se explican los conceptos necesarios para construir esa solución. En el capítulo 16 encontrará el código completo del caso, comentado línea por línea, que es el mismo que se desarrolla en la práctica guiada del módulo.

CAPÍTULO 2

¿Qué es automatizar?

Automatizar en Excel significa lograr que el programa ejecute por usted una serie de pasos que antes realizaba manualmente. Usted define el procedimiento una vez y luego lo reutiliza todas las veces que lo necesite, con la seguridad de que los pasos se harán siempre en el mismo orden y de la misma manera.

Beneficios

- **Ahorro de tiempo:** lo que tomaba horas se hace en segundos.
- **Menos errores:** los pasos no se olvidan ni se alteran por cansancio o distracción.

- **Estandarización:** todos los informes salen con el mismo formato y estructura, lo que facilita compararlos.
- **Más tiempo para analizar:** el personal dedica su esfuerzo a interpretar los resultados en vez de prepararlos.

¿Cuándo conviene automatizar?

Conviene automatizar	No conviene automatizar
Tareas que se repiten (diarias, semanales, mensuales).	Tareas que se hacen una sola vez.
Pasos claros, que siempre son iguales.	Decisiones que dependen del criterio de una persona.
Mucho volumen de datos.	Procesos que cambian cada vez que se realizan.

> **RECUERDE**
> Una macro hace exactamente lo que se le indica. Si el procedimiento está mal pensado, la macro repetirá el error muy rápido. Por eso, primero se diseña el proceso y después se automatiza.

CAPÍTULO 3

Macros y VBA.

Una macro es una secuencia de instrucciones guardada dentro del libro. VBA es el lenguaje en el que se escriben esas instrucciones.

Término	Significado
Macro	Conjunto de instrucciones que Excel ejecuta de forma automática.
VBA	*Visual Basic para Aplicaciones*: lenguaje de programación incluido en Excel y en las demás aplicaciones de Office.
Módulo	Contenedor dentro del libro donde se guarda el código.
Procedimiento	Cada macro; se escribe entre `Sub` y `End Sub`.

Dos formas de crear una macro

Una macro puede **grabarse** (Excel anota las acciones que usted realiza con el ratón y el teclado) o **escribirse** directamente en VBA. Ambas producen el mismo tipo de código; la diferencia está en la flexibilidad.

Macro grabada	Macro escrita en VBA
Excel anota sus clics y teclas.	Usted escribe las instrucciones.
No requiere saber programar.	Requiere conocer el lenguaje.
Repite siempre exactamente lo mismo.	Puede tomar decisiones (`If`) y repetir tareas (`For`).
Útil para empezar y para tareas simples.	Más flexible y fácil de mantener.

> **EN LA PRÁCTICA**
> Lo habitual es grabar una macro para obtener el código básico y luego abrir el editor de VBA para mejorarlo: quitar pasos innecesarios, agregar comentarios y añadir decisiones o repeticiones.

CAPÍTULO 4

Preparar Excel para trabajar con macros.

4.1 Mostrar la pestaña Programador

Las herramientas de macros están en la pestaña **Programador**, que Excel oculta por defecto. Para mostrarla: `Archivo › Opciones › Personalizar cinta de opciones` y marque la casilla **Programador**. Allí encontrará *Grabar macro*, *Macros*, *Visual Basic* e *Insertar*. Este paso se hace una sola vez en cada computadora.

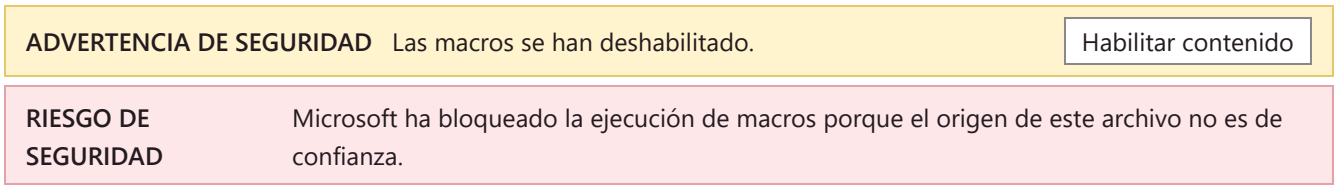
4.2 Guardar como libro habilitado para macros

El formato habitual de Excel (.xlsx) no admite macros. Si guarda un libro con código en ese formato, Excel elimina el código. Por eso, use `Archivo › Guardar como` y elija el tipo **Libro de Excel habilitado para macros (*.xlsm)**.

Extensión	¿Guarda macros?
`.xlsx`	No. Al guardar, Excel elimina el código.
`.xlsm`	Sí. Libro habilitado para macros (el más usado).
`.xlsb`	Sí. Libro binario, útil para archivos muy grandes.

4.3 Seguridad de las macros

Una macro puede modificar o borrar información del libro o de la computadora. Por eso Excel no las ejecuta hasta que usted lo autoriza, y muestra uno de estos avisos:



- Pulse **Habilitar contenido** solo si conoce y confía en el origen del archivo.
- Si el archivo llegó por internet o por correo y aparece el aviso rojo: cierre Excel, haga clic derecho en el archivo › **Propiedades** › marque **Desbloquear** › **Aceptar**.
- La configuración general está en `Archivo › Opciones › Centro de confianza › Configuración del Centro de confianza › Configuración de macros`. La opción recomendada es deshabilitar las macros **con notificación**.

CAPÍTULO 5

Grabar macros.

El grabador de macros convierte sus acciones en código VBA. Es la forma más sencilla de crear una primera macro.

Pasos para grabar una macro

1. `Programador › Grabar macro`.
2. Escriba el nombre de la macro, elija **Guardar macro en: Este libro** y pulse **Aceptar**. Desde ese momento Excel está grabando.
3. Realice las acciones: seleccionar celdas, aplicar formatos, escribir datos, etc.
4. `Programador › Detener grabación`.
5. Para ejecutarla: `Programador › Macros`, elija la macro y pulse **Ejecutar**.

El cuadro «Grabar macro»

Campo	Qué indica
`Nombre de la macro`	Debe empezar con una letra y no puede tener espacios. Puede usar guion bajo. Ejemplo: `FormatoEncabezado`.
`Método abreviado`	Combinación **Ctrl + letra** para ejecutarla desde el teclado (opcional).
`Guardar macro en`	**Este libro** (la macro viaja con el archivo), **Libro nuevo** o **Libro de macros personal** (queda disponible en todos los libros de esa computadora).
`Descripción`	Texto opcional que explica para qué sirve la macro.

Referencias absolutas y relativas

Por defecto, el grabador usa **referencias absolutas**: la macro actúa siempre sobre las mismas celdas (por ejemplo, `B2`), sin importar dónde esté el cursor. Si activa `Programador › Usar referencias relativas` antes de grabar, la macro actúa en relación con la celda activa (por ejemplo, «una celda a la derecha»). Para informes con una estructura fija, como el del caso, conviene la referencia absoluta.

> IMPORTANTE
> Mientras graba, todo queda anotado, incluso los errores y los clics de más. Planifique los pasos antes de empezar. Además, el grabador no puede crear decisiones ni repeticiones: para eso hay que escribir código.

CAPÍTULO 6

El editor de VBA.

El editor de Visual Basic es el lugar donde se leen, escriben y corrigen las macros. Se abre con **Alt + F11** o con `Programador › Visual Basic`. Para escribir una macro nueva se agrega un módulo: `Insertar › Módulo`.

Ventana	Para qué sirve
`Explorador de proyectos (Ctrl + R)`	Muestra los libros abiertos con sus hojas y módulos.
`Propiedades (F4)`	Permite cambiar propiedades del objeto seleccionado, por ejemplo el nombre de una hoja.
`Código`	Donde se escriben y leen los procedimientos.
`Inmediato (Ctrl + G)`	Sirve para probar instrucciones y ver los mensajes de `Debug.Print`.

Formas de ejecutar una macro

- Desde el editor: coloque el cursor dentro de la macro y pulse **F5**.
- Desde Excel: `Programador › Macros` › elija la macro › **Ejecutar**.
- Con un método abreviado (Ctrl + letra) o con un botón asignado (capítulo 15).

CAPÍTULO 7

La estructura de una macro.

Todo procedimiento empieza con `Sub` y termina con `End Sub`. Lo que está entre ambas líneas son las instrucciones que se ejecutan, de arriba hacia abajo.

```
' Muestra un saludo al usuario
Sub Saludar()
    MsgBox "¡Hola, equipo!"      ' instrucción que se ejecuta
End Sub
```

Parte	Función
`Sub Saludar()`	Inicio y nombre de la macro. Los paréntesis son obligatorios.
`MsgBox "…"`	Instrucción que se ejecuta: muestra un mensaje.
`End Sub`	Fin del procedimiento.
`' texto`	Comentario: Excel no lo ejecuta y lo muestra en verde.

Reglas para escribir código claro

- **Nombres descriptivos**, sin espacios ni tildes: `GenerarInforme`, `DarFormato`.
- **Sangría** (tecla Tab) dentro de cada bloque, para ver de un vistazo dónde empieza y termina.
- **Comentarios** con apóstrofo para explicar qué hace cada parte del código.
- Una macro grande puede **llamar a otras macros** escribiendo su nombre. Así el trabajo se divide en partes pequeñas, fáciles de probar.

AYUDA DEL EDITOR

Al terminar una línea, el editor corrige las mayúsculas de las palabras que reconoce (`msgbox` pasa a `MsgBox`) y marca en rojo las líneas con errores de escritura. Si una palabra no cambia a mayúscula, probablemente está mal escrita.

CAPÍTULO 8

El modelo de objetos de Excel.

Para VBA, Excel está formado por **objetos** organizados unos dentro de otros: la aplicación contiene libros, cada libro contiene hojas y cada hoja contiene rangos de celdas. Para actuar sobre algo, se nombra el objeto de lo general a lo particular, separando con puntos.



Figura 2. Jerarquía de objetos de Excel.

```
' Escribe un texto en la celda A1 de la hoja Datos de este libro
ThisWorkbook.Worksheets("Datos").Range("A1").Value = "Reclamación"
```

Concepto	Qué es	Ejemplo
Objeto	Un elemento de Excel.	`Range("A1")`, `Worksheets("Datos")`
Colección	El conjunto de objetos del mismo tipo.	`Worksheets` (todas las hojas)
Propiedad	Una característica: cómo es el objeto.	`.Value`, `.Name`, `.Font.Bold`
Método	Una acción: qué hace el objeto.	`.Delete`, `.Clear`, `.Activate`

ThisWorkbook frente a ActiveSheet

`ThisWorkbook` es el libro donde está guardada la macro, y por eso es la referencia más segura. `ActiveWorkbook` y `ActiveSheet` son el libro y la hoja que estén activos en ese momento, y cambian según dónde haya hecho clic el usuario. Se recomienda nombrar las hojas explícitamente, por ejemplo `ThisWorkbook.Worksheets("Datos")`, para que la macro funcione aunque el usuario esté en otra hoja.

> **ERROR FRECUENTE**
> «Subíndice fuera del intervalo» aparece cuando se nombra una hoja que no existe o cuyo nombre está mal escrito (por ejemplo, «Dato» en lugar de «Datos»).

CAPÍTULO 9

Trabajar con celdas: Range y Cells.

Las dos formas más usadas de referirse a celdas son `Range`, que usa la dirección, y `Cells`, que usa números de fila y columna.

```
Range("A1").Value = "Informe mensual"    ' escribe un texto
Range("B2").Value = 100                  ' escribe un número
Cells(5, 3).Value = 250                  ' fila 5, columna 3: celda C5
Range("A1:C1").Font.Bold = True          ' negrita en un rango
total = Range("F2").Value                ' lee el valor de F2
```

`Range("A1")` es cómodo cuando la celda es fija. `Cells(fila, columna)` es ideal cuando la fila o la columna cambian, por ejemplo dentro de una repetición: `Cells(5, 3)` es la celda C5.

Propiedad o método	Qué hace
`.Value`	Lee o escribe el valor de la celda.
`.Formula`	Lee o escribe una fórmula (escrita en inglés).
`.NumberFormat`	Formato de número, por ejemplo `"#,##0.00"` o `"""RD$"" #,##0.00"`.
`.Font.Bold / .Size / .Color`	Negrita, tamaño y color de la letra.
`.Interior.Color`	Color de relleno, con `RGB(rojo, verde, azul)`.
`.Borders.LineStyle`	Bordes del rango, por ejemplo `xlContinuous`.
`.ClearContents / .Clear`	Borra solo el contenido / borra contenido y formato.
`.Columns(…).AutoFit`	Ajusta el ancho de las columnas al contenido.

WITH … END WITH
Cuando varias instrucciones se aplican al mismo objeto, `With` evita repetir su nombre: dentro del bloque, cada línea que empieza con punto se refiere a ese objeto. Lo verá en la macro DarFormato del capítulo 16.

CAPÍTULO 10

Variables y tipos de datos.

Una **variable** es un espacio con nombre donde la macro guarda un dato mientras se ejecuta. Se declara con `Dim`, indicando el tipo de dato, y luego se le asigna un valor con el signo igual. Los objetos (hojas, rangos) se asignan con `Set`.

```vba
Dim total As Double          ' número con decimales
Dim ars As String            ' texto
Dim fila As Integer          ' número entero
Dim datos As Worksheet       ' una hoja (objeto)

total = 0
ars = "ARS Senasa"
Set datos = ThisWorkbook.Worksheets("Datos")    ' los objetos usan Set
```

Tipo	Guarda	Ejemplo
Integer	Números enteros pequeños (hasta 32,767).	`fila = 5`
Long	Números enteros grandes.	`n = 150000`
Double	Números con decimales.	`monto = 2950.5`
String	Texto.	`ars = "ARS Humano"`
Boolean	Verdadero o falso.	`listo = True`
Date	Fechas.	`hoy = Date`
Variant	Cualquier tipo de dato, incluso listas.	`lista = Array("A", "B")`

OPTION EXPLICIT

Si escribe `Option Explicit` en la primera línea del módulo, VBA exige declarar todas las variables y avisa cuando una está mal escrita. Puede activarlo para todos los módulos nuevos en el editor: Herramientas › Opciones › «Requerir declaración de variables».

CAPÍTULO 11

Decisiones: If...Then...Else.

Con `If` la macro evalúa una condición y elige qué hacer según el resultado.

```
If Range("C9").Value > 500000 Then
    MsgBox "El monto supera el presupuesto"
ElseIf Range("C9").Value > 400000 Then
    MsgBox "El monto está cerca del límite"
Else
    MsgBox "Monto dentro del presupuesto"
End If
```

VBA revisa las condiciones de arriba hacia abajo y ejecuta solo el primer bloque cuya condición se cumple. `ElseIf` y `Else` son opcionales. Una decisión corta puede escribirse en una sola línea, sin `End If`: `If hoja.Name = "Reporte" Then hoja.Delete`.

Operador	Significado
`=`	igual a
`<>`	distinto de
`> · <`	mayor que · menor que
`>= · <=`	mayor o igual · menor o igual
`And`	se cumplen las dos condiciones
`Or`	se cumple al menos una condición

> IMPORTANTE
> Todo `If` escrito en varias líneas debe cerrar con `End If`. Si falta, el editor muestra «Bloque If sin End If».

CAPÍTULO 12

Repeticiones: For...Next y For Each.

For...Next: repetir un número de veces

```
Dim fila As Integer
For fila = 2 To 121                        ' las 120 reclamaciones
    If Cells(fila, 6).Value > 20000 Then   ' columna 6 = F (Monto)
        Cells(fila, 6).Interior.Color = RGB(255, 230, 150)    ' amarillo
    End If
Next fila
```

La variable `fila` toma los valores 2, 3, 4... hasta 121. En cada vuelta se revisa el monto de la columna F (columna 6) y, si es mayor de RD$ 20,000, se marca en amarillo. Una persona tardaría varios minutos en revisar 120 filas; la macro lo hace en un segundo.

For Each...Next: recorrer una colección

```
Dim hoja As Worksheet
For Each hoja In ThisWorkbook.Worksheets    ' cada hoja del libro
    hoja.Range("A1").Font.Bold = True
Next hoja
```

Recorre todas las hojas del libro, sin importar cuántas sean. Es la forma natural de trabajar con colecciones (hojas, celdas de un rango).

Use	Cuando
`For…Next`	Sabe desde qué número hasta cuál repetir (filas, columnas, un contador).
`For Each`	Quiere recorrer todos los elementos de una colección.

CAPÍTULO 13

Hojas y fórmulas desde VBA.

13.1 Crear, limpiar y eliminar hojas

```
Worksheets.Add.Name = "Reporte"          ' crea una hoja y le pone nombre
Worksheets("Reporte").Cells.Clear        ' la deja en blanco
Worksheets("Datos").Activate             ' muestra la hoja Datos

Application.DisplayAlerts = False        ' no preguntar al borrar
Worksheets("Reporte").Delete             ' elimina la hoja
Application.DisplayAlerts = True         ' vuelve a mostrar los avisos
```

`Application.DisplayAlerts = False` evita que Excel pregunte «¿Desea eliminar esta hoja?»; siempre debe volver a ponerse en `True` al terminar. Si ya existe una hoja con el nombre que se quiere crear, `Worksheets.Add.Name` produce un error; por eso, antes de crearla, se elimina o se limpia la anterior.

13.2 Fórmulas y funciones

```
' Fórmula viva en la celda (en VBA se escribe en inglés)
Range("C9").Formula = "=SUM(C5:C8)"      ' en la celda se ve =SUMA(C5:C8)

' Resultado calculado por VBA y guardado como valor
n = Application.WorksheetFunction.CountIf(Range("C:C"), "ARS Senasa")
m = Application.WorksheetFunction.SumIf(Range("C:C"), "ARS Senasa", Range("F:F"))
```

- En VBA, las funciones de Excel se escriben **en inglés** (SUM, SUMIF, COUNTIF) y los argumentos se separan con **coma**. Excel las traduce al mostrarlas en la celda.
- `.Formula` deja una fórmula viva en la celda; `WorksheetFunction` calcula el resultado y lo escribe como valor.

En la celda (español)	En VBA (inglés)
SUMA	SUM
CONTAR.SI	CountIf
SUMAR.SI	SumIf
PROMEDIO	Average

> **VELOCIDAD**
> `Application.ScreenUpdating = False` evita que la pantalla se actualice mientras la macro trabaja: el proceso es más rápido y no parpadea. Al terminar se vuelve a poner en `True`.

CAPÍTULO 14

Comunicarse con el usuario: MsgBox e InputBox.

```
mes = InputBox("¿De qué mes es el informe?")

MsgBox "Informe de " & mes & " generado.", _
       vbInformation, "Informe mensual"

resp = MsgBox("¿Desea continuar?", vbYesNo + vbQuestion)
If resp = vbNo Then Exit Sub
```

Elemento	Uso
MsgBox	Muestra un mensaje y espera a que el usuario pulse un botón.
InputBox	Muestra un cuadro para escribir un dato y lo devuelve como texto.
vbInformation · vbExclamation · vbCritical	Icono del mensaje: información, advertencia o error.
vbYesNo	Botones Sí y No. El resultado es `vbYes` o `vbNo`.
&	Une textos y valores en una sola frase.
_ (guion bajo)	Continúa una instrucción larga en la línea siguiente (va precedido de un espacio).
Exit Sub	Termina la macro en ese punto.

CAPÍTULO 15

Ejecutar con un botón y depurar errores.

15.1 Crear un botón

1. `Insertar › Ilustraciones › Formas` y dibuje una forma, por ejemplo un rectángulo con esquinas redondeadas.
2. Escriba dentro el texto del botón, por ejemplo **Generar informe**.
3. Haga clic derecho en el borde de la forma › **Asignar macro...** › elija la macro › **Aceptar**.
4. Haga clic en cualquier celda y luego pulse el botón para comprobar que funciona.

Otra opción es `Programador › Insertar › Botón (control de formulario)`: al dibujarlo, Excel pide directamente la macro que debe ejecutar.

15.2 Depurar: encontrar y corregir errores

Herramienta	Para qué sirve
`F8`	Ejecuta la macro línea por línea (paso a paso) y resalta en amarillo la línea siguiente.
`F9`	Pone o quita un punto de interrupción: la macro se detiene al llegar a esa línea.
`Debug.Print`	Escribe valores en la ventana Inmediato para revisar qué está pasando.

Mensaje de error	Causa probable	Qué hacer
Subíndice fuera del intervalo	La hoja o el elemento no existe o su nombre está mal escrito.	Revise el nombre exacto de la hoja.
Bloque If sin End If	Falta cerrar una decisión.	Agregue `End If`.
Sub o Function no definida	Se llamó a una macro que no existe o está mal escrita.	Revise el nombre de la macro.
Las macros no se ejecutan	El archivo es .xlsx o el contenido no está habilitado.	Guarde como .xlsm y habilite el contenido.

> **CONSEJO**
> Lea el mensaje de error con calma: casi siempre indica qué falta. Al pulsar **Depurar**, el editor resalta en amarillo la línea donde se detuvo la macro.

CAPÍTULO 16

Caso integrador: el informe mensual automatizado.

Una macro principal, `GenerarInforme`, llama en orden a tres macros más pequeñas. Dividir el trabajo así hace el código más fácil de leer, probar y corregir.

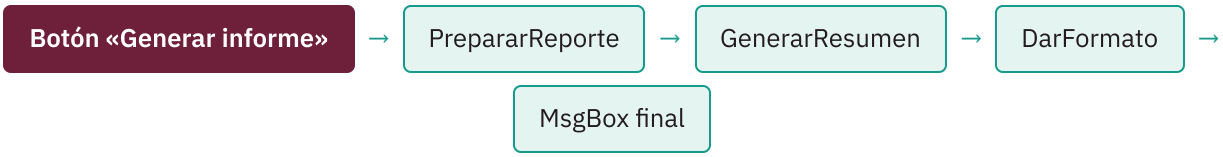


Figura 3. Flujo del informe automatizado.

El libro tiene una hoja llamada **Datos** con 120 reclamaciones de octubre de 2026 (datos ficticios): en la columna C está la ARS y en la columna F, el monto. La macro crea la hoja **Reporte**, calcula la cantidad y el monto de cada ARS, le da formato y avisa al terminar. Este es el código completo, comentado:

```
'====================================================================
' Módulo 7 · Práctica guiada · Informe mensual automatizado
' Hoja de origen: Datos (A: Reclamación, C: ARS, F: Monto)
'====================================================================

' PASO 4 · Crea (o vuelve a crear) la hoja Reporte con sus títulos
Sub PrepararReporte()
    Dim hoja As Worksheet                    ' variable que guarda una hoja

    ' 1. Si ya existe una hoja llamada Reporte, se borra
    Application.DisplayAlerts = False        ' no pedir confirmación al borrar
    For Each hoja In ThisWorkbook.Worksheets  ' recorre todas las hojas del libro
        If hoja.Name = "Reporte" Then hoja.Delete
    Next hoja
    Application.DisplayAlerts = True         ' vuelve a mostrar los avisos

    ' 2. Agrega una hoja nueva al final y le pone nombre
    Set hoja = ThisWorkbook.Worksheets.Add( _
        After:=ThisWorkbook.Worksheets(ThisWorkbook.Worksheets.Count))
    hoja.Name = "Reporte"

    ' 3. Escribe el título, la fecha y los encabezados
    hoja.Range("A1").Value = "Informe mensual de reclamaciones"
    hoja.Range("A2").Value = "Generado el " & Format(Date, "dd/mm/yyyy")
    hoja.Range("A4").Value = "ARS"
    hoja.Range("B4").Value = "Reclamaciones"
    hoja.Range("C4").Value = "Monto"
End Sub

' PASO 5 · Calcula cuántas reclamaciones y cuánto monto tiene cada ARS
Sub GenerarResumen()
    Dim datos As Worksheet, rep As Worksheet  ' hoja de origen y hoja de reporte
    Dim listaARS As Variant                   ' lista con los nombres de las ARS
```

```
    Dim i As Integer, fila As Integer          ' contador y fila donde se escribe

    Set datos = ThisWorkbook.Worksheets("Datos")
    Set rep = ThisWorkbook.Worksheets("Reporte")
    listaARS = Array("ARS Humano", "ARS Primera", "ARS Senasa", "ARS Universal")

    fila = 5                                   ' los resultados empiezan en la fila 5
    For i = 0 To UBound(listaARS)              ' repite una vez por cada ARS
        rep.Cells(fila, 1).Value = listaARS(i)
        rep.Cells(fila, 2).Value = Application.WorksheetFunction.CountIf( _
            datos.Range("C:C"), listaARS(i))                ' CONTAR.SI
        rep.Cells(fila, 3).Value = Application.WorksheetFunction.SumIf( _
            datos.Range("C:C"), listaARS(i), datos.Range("F:F"))  ' SUMAR.SI
        fila = fila + 1                        ' baja a la fila siguiente
    Next i

    ' Fila de total (fila 9): en VBA las fórmulas se escriben en inglés
    rep.Range("A9").Value = "Total"
    rep.Range("B9").Formula = "=SUM(B5:B8)"
    rep.Range("C9").Formula = "=SUM(C5:C8)"
End Sub

' PASO 6 · Da formato al reporte
Sub DarFormato()
    With ThisWorkbook.Worksheets("Reporte")    ' todo lo que empieza con punto es de esta
hoja
        .Range("A1").Font.Bold = True
        .Range("A1").Font.Size = 16
        .Range("A4:C4").Font.Bold = True
        .Range("A4:C4").Font.Color = RGB(255, 255, 255)        ' letra blanca
        .Range("A4:C4").Interior.Color = RGB(110, 31, 58)      ' relleno burdeos
        .Range("C5:C9").NumberFormat = """RD$"" #,##0.00"       ' formato de moneda
        .Range("A9:C9").Font.Bold = True
        .Range("A4:C9").Borders.LineStyle = xlContinuous       ' bordes en la tabla
        .Columns("A:C").AutoFit                                ' ajusta el ancho
    End With
End Sub

' PASO 7 · Ejecuta todo el proceso y avisa al terminar
Sub GenerarInforme()
    Application.ScreenUpdating = False         ' trabaja sin parpadeos
    PrepararReporte                            ' 1. hoja nueva con títulos
    GenerarResumen                             ' 2. cálculos por ARS
    DarFormato                                 ' 3. formatos
    Application.ScreenUpdating = True

    ThisWorkbook.Worksheets("Reporte").Activate
    MsgBox "Informe generado." & vbCrLf & "Monto total: RD$ " & _
        Format(ThisWorkbook.Worksheets("Reporte").Range("C9").Value, "#,##0.00"), _
```

```
        vbInformation, "Informe mensual"
End Sub
```

Resultado esperado

ARS	Reclamaciones	Monto
ARS Humano	35	RD$ 166,200.00
ARS Primera	28	RD$ 141,650.00
ARS Senasa	38	RD$ 225,900.00
ARS Universal	19	RD$ 160,100.00
Total	**120**	**RD$ 693,850.00**

Al terminar aparece el mensaje «Informe generado. Monto total: RD$ 693,850.00». Si la macro se ejecuta de nuevo, la hoja Reporte se vuelve a crear: nunca se duplica.

CAPÍTULO 17

Buenas prácticas y aplicación institucional.

Buenas prácticas

1. **Guarde una copia** del archivo antes de probar una macro: los cambios que hace una macro no se pueden deshacer con Ctrl + Z.
2. **Comente** el código para que otras personas, o usted mismo dentro de unos meses, entiendan qué hace.
3. Use **nombres claros** en macros, variables y hojas.
4. **Pruebe paso a paso** (F8) con datos de ejemplo antes de usar la macro con datos reales.
5. Divida los procesos largos en **macros pequeñas** que se llaman desde una principal.
6. Habilite **solo macros de confianza** y no comparta archivos con código que no conoce.

Aplicación en una institución como SISALRIL

Las instituciones del sector salud manejan reportes periódicos con gran volumen de información, por ejemplo las reclamaciones y los servicios reportados por las ARS. Automatizar las tareas repetitivas de esos reportes permite:

Beneficio	Ejemplo
Menos tiempo	Consolidar y resumir reportes de varias ARS en minutos, en lugar de horas.
Menos errores	Cálculos y formatos idénticos cada mes, sin depender de la memoria de quien prepara el informe.
Informes comparables	El mismo formato en todos los periodos facilita detectar cambios y tendencias.
Más análisis	El personal dedica su tiempo a interpretar los datos y proponer decisiones.

> **REFLEXIÓN**
> La macro prepara la información; las decisiones siguen siendo responsabilidad de las personas. Una buena automatización libera tiempo para pensar, no reemplaza el criterio profesional.

CAPÍTULO 18

Glosario.

Término	Definición
Colección	Conjunto de objetos del mismo tipo, como `Worksheets`.
Comentario	Texto precedido de apóstrofo que explica el código y no se ejecuta.
Depurar	Encontrar y corregir errores en una macro.
Macro	Secuencia de instrucciones guardada que Excel ejecuta automáticamente.
Método	Acción que puede realizar un objeto, como `.Delete`.
Módulo	Contenedor del código dentro del libro.
Objeto	Elemento de Excel que se puede manipular: libro, hoja, rango.
Procedimiento	Macro escrita entre `Sub` y `End Sub`.
Propiedad	Característica de un objeto, como `.Value` o `.Name`.
Variable	Espacio con nombre que guarda un dato mientras se ejecuta la macro.
VBA	Visual Basic para Aplicaciones, el lenguaje de las macros de Office.
.xlsm	Formato de libro de Excel que guarda macros.

Referencias.

Alexander, M., & Kusleika, D. (2019). *Excel 2019 power programming with VBA*. Wiley.

Figueroa, J. (2026). *Base de reclamaciones del Módulo 7* [Conjunto de datos ficticio]. Instituto ICADEP-XXI.

Microsoft. (s. f.-a). *Habilitar o deshabilitar macros en archivos de Microsoft 365*. Microsoft Support. https://support.microsoft.com/es-ES/Office/vba/enable-or-disable-macros-in-microsoft-365-files

Microsoft. (s. f.-b). *Hacer referencia a las hojas por nombre*. Microsoft Learn. https://learn.microsoft.com/es-es/office/vba/excel/concepts/workbooks-and-worksheets/refer-to-sheets-by-name

Microsoft. (s. f.-c). *Propiedad Application.ShowDevTools (Excel)*. Microsoft Learn. https://learn.microsoft.com/es-es/office/vba/api/excel.application.showdevtools

Microsoft. (s. f.-d). *Protéjase de los virus de macros*. Microsoft Support. https://support.microsoft.com/es-ES/Office/add-ins/protect-yourself-from-macro-viruses

Microsoft. (s. f.-e). *Referencia de Visual Basic para Aplicaciones (VBA) de Excel*. Microsoft Learn. https://learn.microsoft.com/es-es/office/vba/api/overview/excel